

MACHINE LEARNING LAB

NAME: **Narayan behera**

ENROLLMENT NO: **A710145025032**

MCA SEM 2

PRACTICAL 1.1:

Using the Pandas library of Python, exhibit the process of data extraction and preparation. Exhibit the following operations:

1. Arrange the below-mentioned data into a table.
2. Divide the dataset into two parts as 70% for training and 30% for testing.

```
data = {  
    "country": ["Brazil", "Russia", "India", "China", "South Africa"],  
    "capital": ["Brasilia", "Moscow", "New Delhi", "Beijing", "Pretoria"],  
    "area": [8.516, 17.10, 3.286, 9.597, 1.221],  
    "population": [200.4, 143.5, 1252, 1357, 52.98]  
}
```

- ☐ Pandas is a Python library used for data manipulation and analysis.
- ☐ It provides data structures like DataFrame to handle structured data easily.

```
import pandas as pd  
data = {  
    "country": ["Brazil", "Russia", "India", "China", "South Africa"],  
    "capital": ["Brasilia", "Moscow", "New Delhi", "Beijing", "Pretoria"],  
    "area": [8.516, 17.10, 3.286, 9.597, 1.221],  
    "population": [200.4, 143.5, 1252, 1357, 52.98]  
}
```

```
df = pd.DataFrame(data)
```

```
print("Full Data:")  
print(df)
```

```

... Full Data:
      country  capital  area  population
0      Brazil  Brasilia  8.516      200.40
1      Russia   Moscow 17.100      143.50
2       India  New Delhi  3.286     1252.00
3       China   Beijing  9.597     1357.00
4  South Africa  Pretoria  1.221       52.98

```

```

from sklearn.model_selection import train_test_split
train, test = train_test_split(df, test_size=0.3,
random_state=42) print("\nTraining Data (70%):")
print(train) print("\nTesting Data (30%):") print(test)

```

```

...
Training Data (70%):
      country  capital  area  population
2      India  New Delhi  3.286     1252.0
0  Brazil   Brasilia  8.516       200.4
3    China   Beijing  9.597     1357.0

Testing Data (30%):
      country  capital  area  population
1      Russia   Moscow 17.100      143.50
4  South Africa  Pretoria  1.221       52.98

```

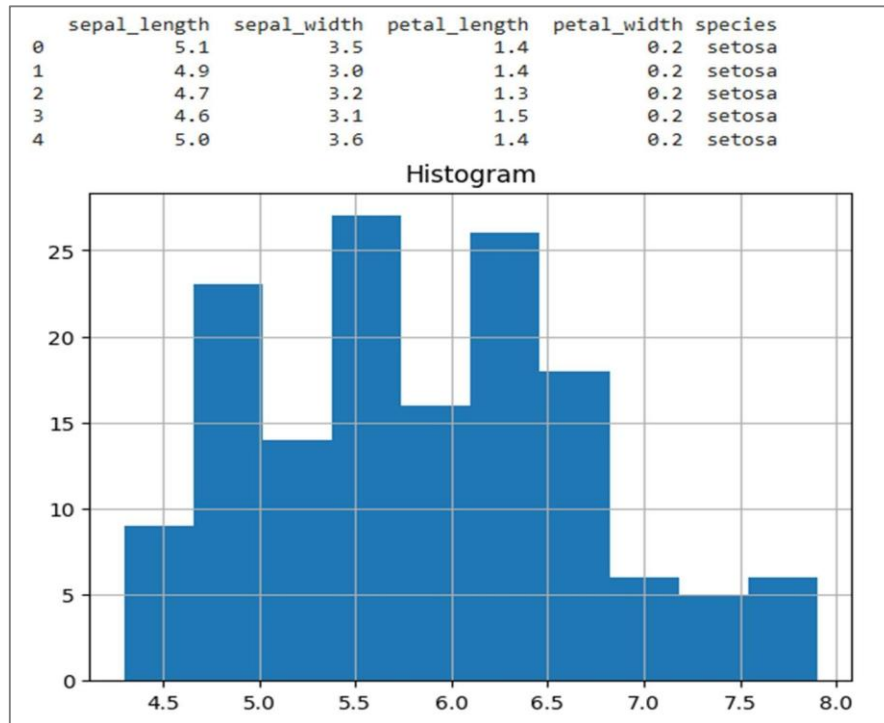
PRACTICAL 1.2

For Data Visualization, univariate and multivariate plotting schemes are used to understand ML data with statistics. Considering appropriate datasets from standard repositories, build pandas dataframes and plot the following plots using matplotlib graph plotting libraries in Python:

1. Histogram
2. Bar Graphs
3. Density Plot
4. Pie Charts
5. Box and Whisker Plot
6. Scatter Plot
7. Correlation Matrix Plot
8. Scatter Matrix Plot

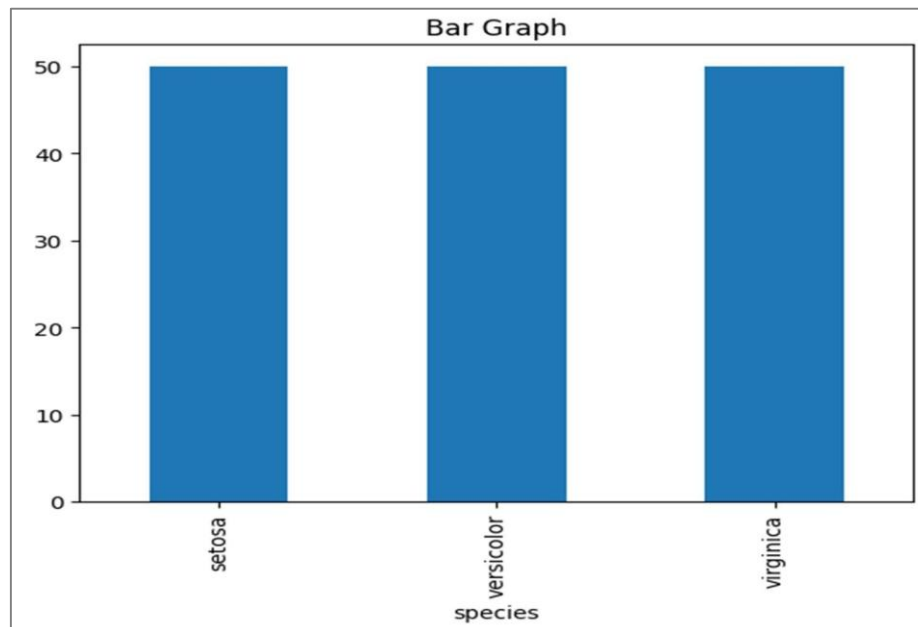
- Matplotlib: A library used for creating static graphs and plots in Python.
- Histogram: A graph showing the frequency distribution of continuous data.
- Bar Graph: A chart that represents categorical data using rectangular bars.
- Density Plot: A smooth curve showing the distribution of data (probability density).
- Pie Chart: A circular chart showing proportion of categories as slices.
- Box and Whisker Plot: A plot that shows median, quartiles, and outliers of data.
- Scatter Plot: A graph showing the relationship between two variables.
- Correlation Matrix: A table showing correlation values between variables.
- Scatter Matrix: A grid of scatter plots showing pairwise relationships between variables.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df =
sns.load_dataset("iris")
print(df.head()) # 1.
Histogram
df["sepal_length"].hist()
plt.title("Histogram")
plt.show()
```



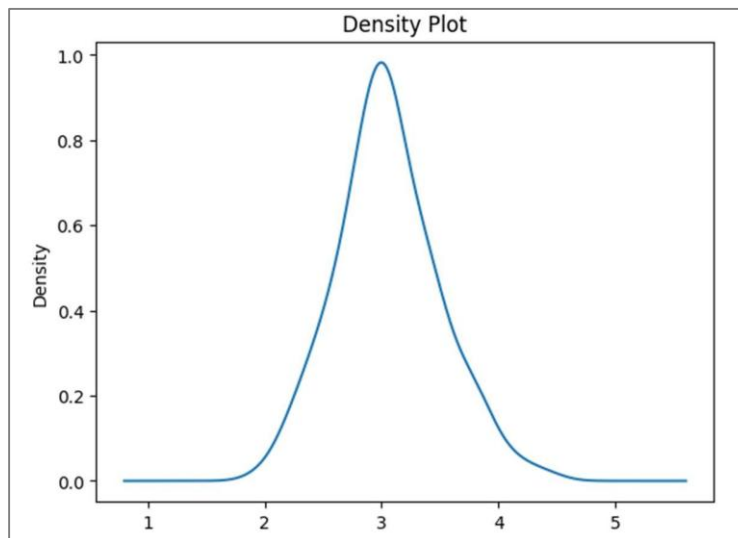
2. Bar Graph

```
df["species"].value_counts().plot(kind='
bar') plt.title("Bar Graph") plt.show()
```

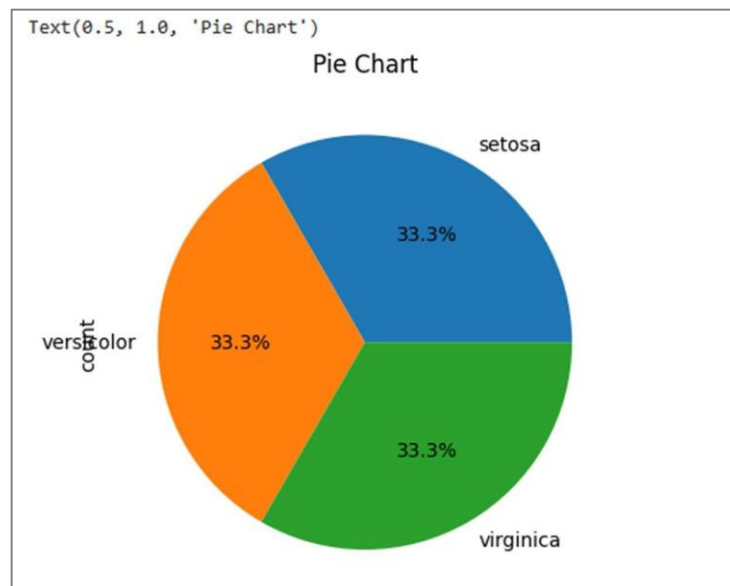


3. Density Plot

```
df["sepal_width"].plot(kind='density')  
plt.title("Density Plot")  
plt.show()
```

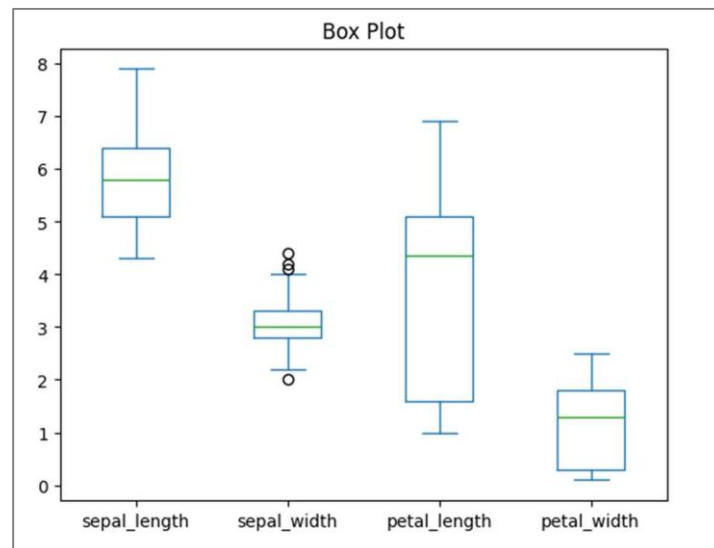


4. Pie Chart df["species"].value_counts().plot(kind='pie', autopct='%1.1f%%') plt.title("Pie Chart")



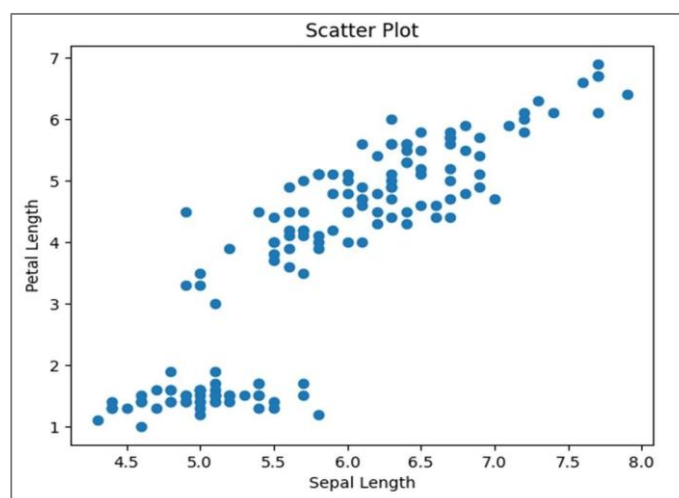
5. Box Plot

```
df.plot(kind='box')  
plt.title("Box Plot")  
plt.show()
```



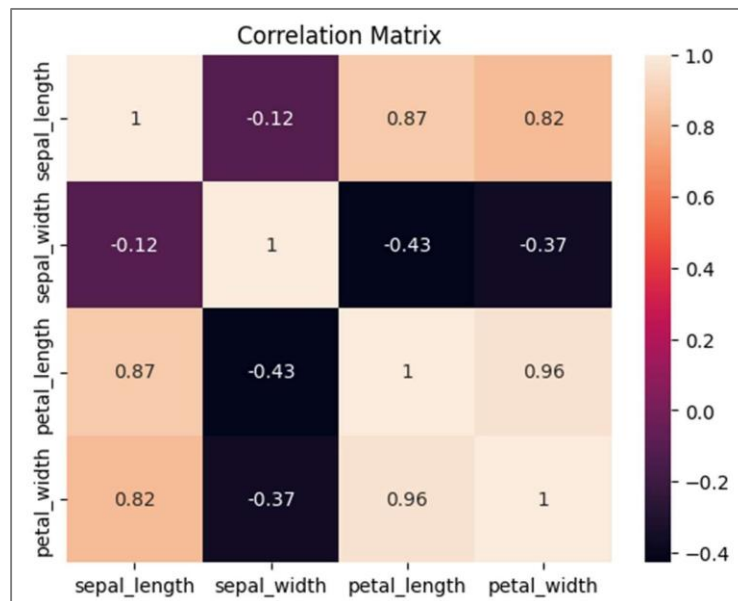
6. Scatter Plot

```
plt.scatter(df["sepal_length"],  
df["petal_length"]) plt.xlabel("Sepal  
Length") plt.ylabel("Petal Length")  
plt.title("Scatter Plot") plt.show()
```



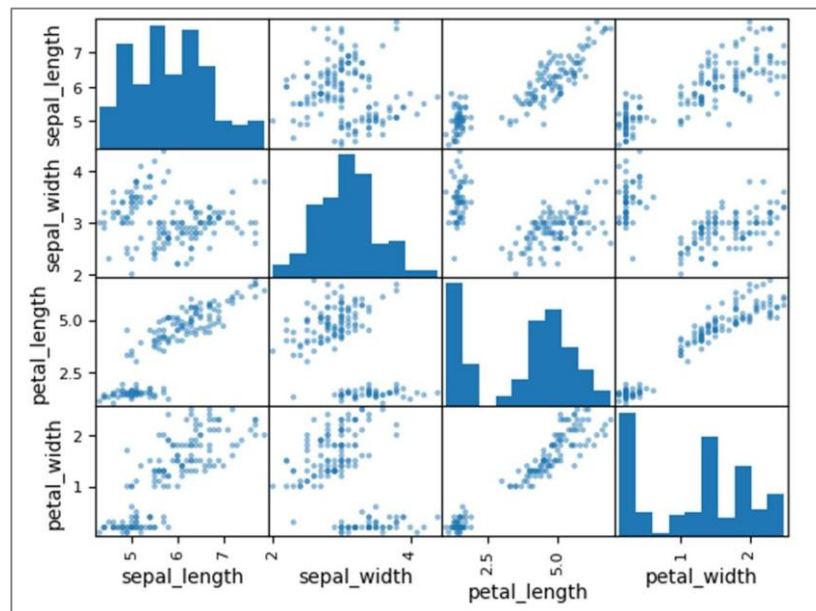
7. Correlation Matrix

```
sns.heatmap(df.corr(numeric_only=True),  
annot=True) plt.title("Correlation Matrix")  
plt.show()
```



8. Scatter Matrix

```
pd.plotting.scatter_matrix(df)  
plt.show()
```



PRACTICAL 1.3

Considering appropriate datasets from standard repositories, plot the following using seaborn graph plotting libraries in Python:

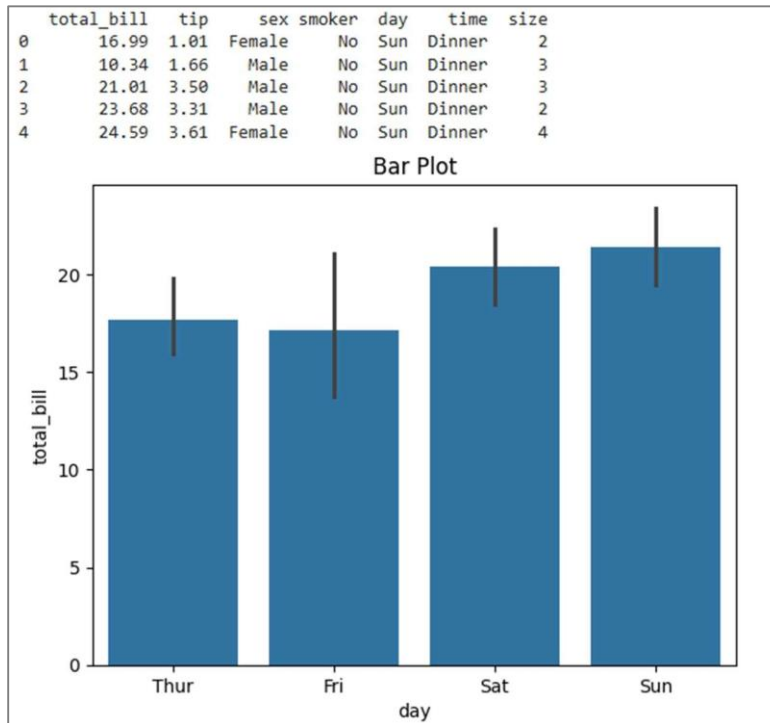
1. Bar Plot
 2. Count Plot
 3. Distribution Plot
 4. Point Plot
 5. Facet Grid
 6. Pair Grid
- Seaborn: A Python library used for advanced and attractive statistical plots.
 - Bar Plot: Shows comparison of values across categories.
 - Count Plot: Displays count/frequency of categorical data.
 - Distribution Plot: Shows distribution of a variable using histogram/KDE.
 - Point Plot: Shows mean values with points and lines.
 - Facet Grid: Creates multiple plots based on subsets of data. □ Pair Grid: Displays pairwise relationships between variables.

```
import seaborn as sns
import matplotlib.pyplot as
plt
```

```
# Load dataset df =
sns.load_dataset("tips")
```

```
print(df.head())
```

```
# 1. Bar Plot sns.barplot(x="day",
y="total_bill", data=df) plt.title("Bar
Plot") plt.show()
```

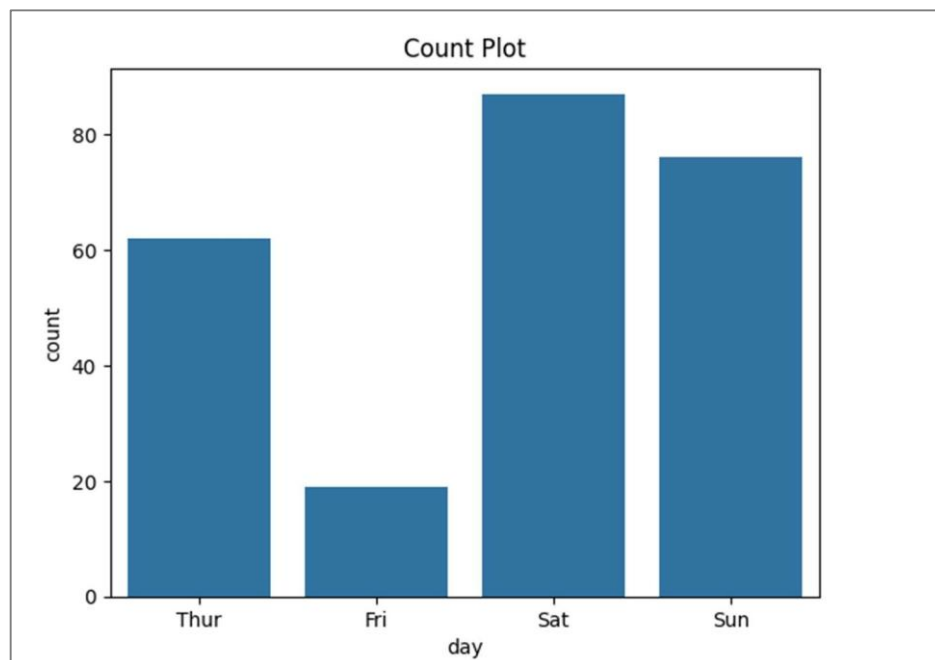


2. Count Plot

```
sns.countplot(x="day", data=df)
```

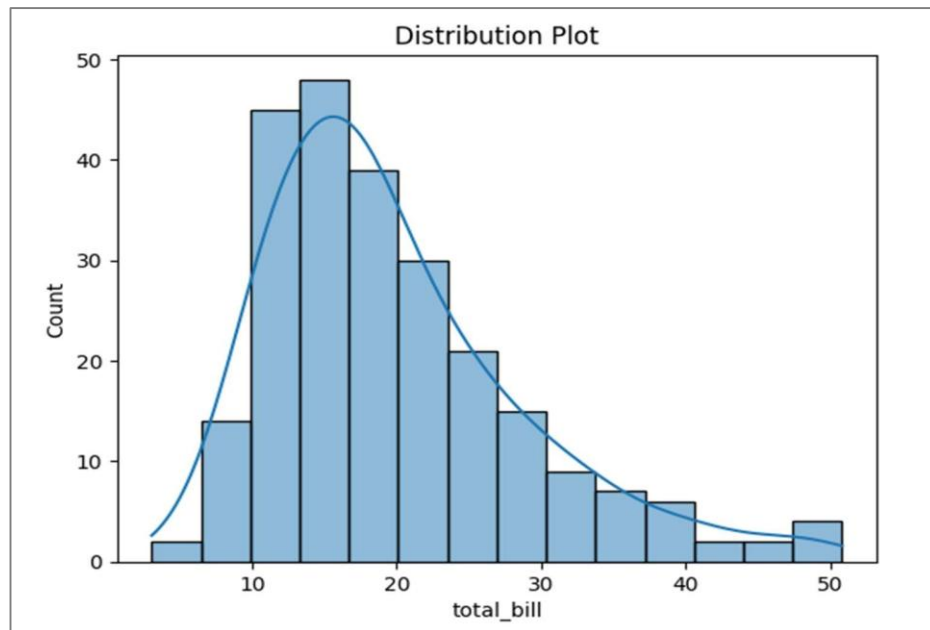
```
plt.title("Count Plot")
```

```
plt.show()
```

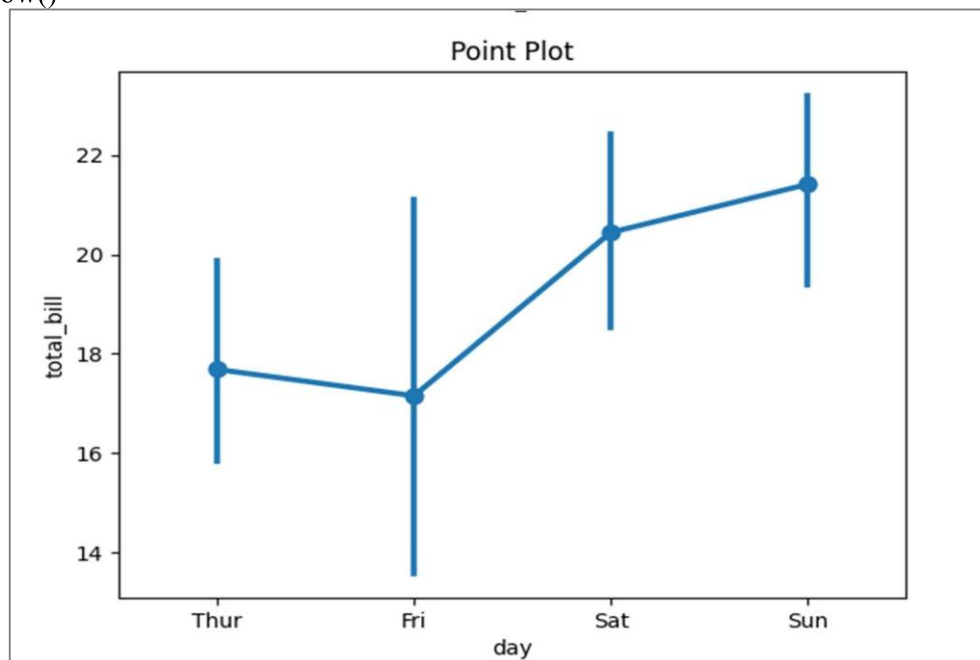


3. Distribution Plot

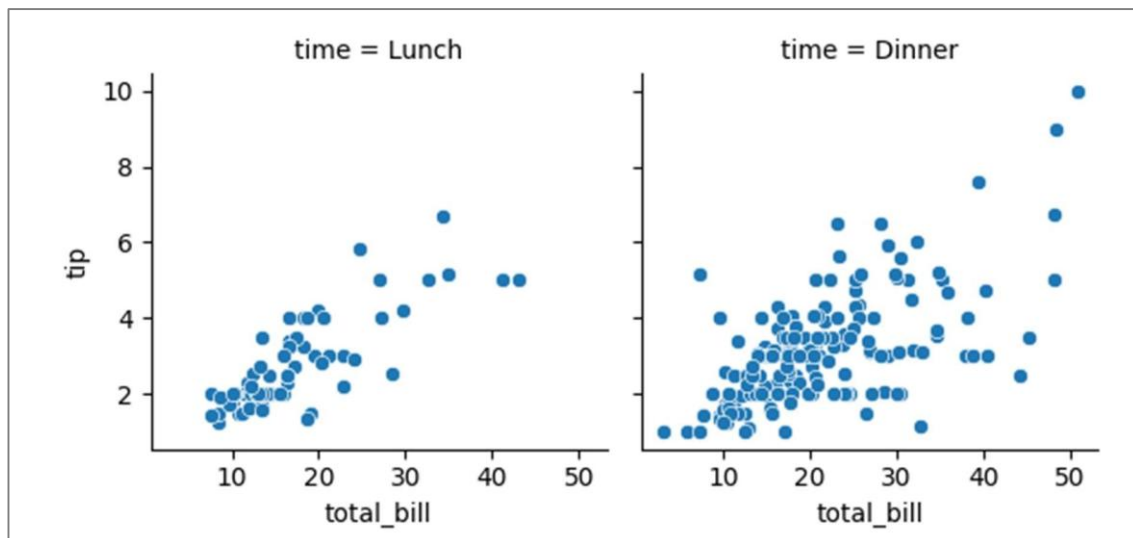
```
sns.histplot(df["total_bill"], kde=True)  
plt.title("Distribution Plot")  
plt.show()
```



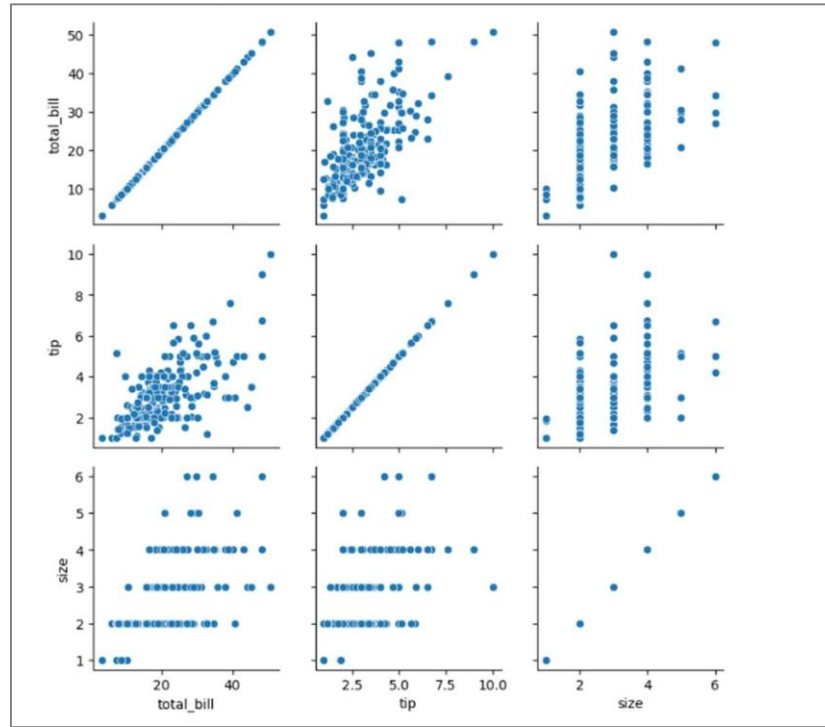
```
# 4. Point Plot  
sns.pointplot(x="day",  
y="total_bill", data=df)  
plt.title("Point Plot")  
plt.show()
```



```
# 5. Facet Grid g =  
sns.FacetGrid(df, col="time")  
g.map(sns.scatterplot, "total_bill", "tip") plt.show()
```



```
# 6. Pair Grid g =  
sns.PairGrid(df)  
g.map(sns.scatterplot)  
plt.show()
```



PRACTICAL 1.4

Using an appropriate dataset from standard repositories, build the decision tree using ID3 algorithmic technique. Further classify a new sample using the model.

- Decision Tree: A tree-like model used for classification and prediction based on decision rules.
- ID3 Algorithm: ID3 (Iterative Dichotomiser 3) is a decision tree algorithm that uses entropy and information gain to split data.
- Entropy: A measure of impurity or randomness in the dataset.

CODE:

```
import pandas as pd
from sklearn.preprocessing import
LabelEncoder from sklearn.tree import
DecisionTreeClassifier

# Dataset
data = {
    'Outlook':
['Sunny','Sunny','Overcast','Rain','Rain','Rain','Overcast','Sunny','Sunny','Rain','Sun
ny','Overca st','Overcast','Rain'],    'Temperature':
['Hot','Hot','Hot','Mild','Cool','Cool','Mild','Hot','Cool','Mild','Mild','Mild','Hot','Mild'],
    'Humidity':
['High','High','High','High','Normal','Normal','Normal','High','Normal','Normal','Normal'
,'High ','Normal','High'],    'Wind':
['Weak','Strong','Weak','Weak','Weak','Strong','Strong','Weak','Weak','Weak','Strong'
,'Strong',' Weak','Strong'],
    'PlayTennis':
['No','No','Yes','Yes','Yes','No','Yes','No','Yes','Yes','Yes','Yes','Yes','No'] }

df = pd.DataFrame(data)

# Encode categorical
data le =
LabelEncoder() for
column in df.columns:
    df[column] = le.fit_transform(df[column])

# Split features and target X
= df.drop('PlayTennis',
axis=1) y = df['PlayTennis']
```

```
# Create Decision Tree using entropy (ID3)
model =
DecisionTreeClassifier(criterion='entropy')
model.fit(X, y)

new_sample = [[2, 0, 0, 1]] # encoded values
prediction = model.predict(new_sample)
print("Prediction (0=No, 1=Yes):", prediction)
```

OUTPUT:

```
Prediction (0=No, 1=Yes): [0]
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not
warnings.warn(
```